

Báo cáo cuối kỳ: Hệ thống cảnh báo đột quy

Source code

Trần Hoàng Ân - 2570548
Trần Huy Phước - 2570482
Phạm Minh Quang - 2570302

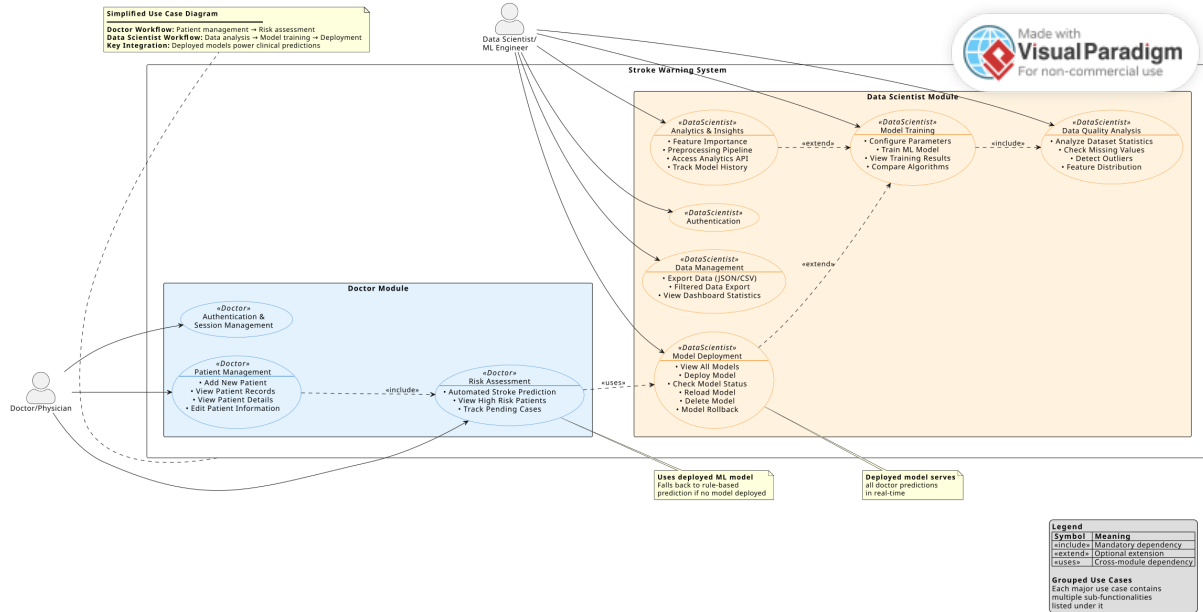
Giảng viên hướng dẫn: PGS.TS. Quản Thành Thơ

Ngày 12 tháng 12 năm 2025

Mục lục

1	Nhiệm vụ của từng Stakeholder	2
1.1	Các bên liên quan được xác định	2
1.2	Các trường hợp sử dụng (Use Cases) của Bác sĩ	2
1.2.1	Xác thực và kiểm soát truy cập	2
1.2.2	Quản lý bệnh nhân	2
1.2.3	Đánh giá rủi ro và hỗ trợ quyết định lâm sàng	3
1.3	Các trường hợp sử dụng (Use Cases) của Nhà khoa học dữ liệu	3
1.3.1	Thu thập và xuất dữ liệu	3
1.3.2	Phân tích và đảm bảo chất lượng dữ liệu	3
1.3.3	Huấn luyện mô hình Machine Learning	3
1.3.4	Quản lý và triển khai mô hình	4
2	Lý thuyết về thuật toán	4
2.1	Hồi quy Logistic (Logistic Regression)	4
2.2	Rừng ngẫu nhiên (Random Forest)	5
2.3	Máy vector hỗ trợ (SVM)	7
2.4	Naive Bayes (GaussianNB)	8
2.5	So sánh tổng quan các thuật toán	9
2.6	Chiến lược lựa chọn thuật toán	9
3	Kiến trúc hệ thống	10
3.1	Thành phần Backend	10
3.2	Thành phần Frontend	10
3.3	Quy trình học máy	11
4	Tập dữ liệu và tiền xử lý	11
4.1	Mô tả tập dữ liệu	11
5	Quy trình huấn luyện và triển khai model	15
5.1	Các bước trong quy trình	15
5.1.1	Bước 1: Thu thập và chuẩn bị dữ liệu	15
5.1.2	Bước 2: Cấu hình và huấn luyện mô hình	15
5.1.3	Bước 3: Đánh giá hiệu suất	16
5.1.4	Bước 4: Lưu trữ và quản lý phiên bản	16
5.1.5	Bước 5: Triển khai mô hình	16
5.1.6	Bước 6: Giám sát và quản lý	17
5.2	Ưu điểm của quy trình	17
6	Kết quả đánh giá model	17
6.1	Các chỉ số đánh giá	17
6.2	Kết quả huấn luyện các mô hình	18
6.2.1	Random Forest	18
6.2.2	Support Vector Machine (SVM)	19
6.2.3	Naive Bayes	19
6.2.4	Logistic Regression	19
6.3	So sánh tổng hợp các mô hình	20

1 Nghiệp vụ của từng Stakeholder



Hình 1: Sơ đồ Use Cases

1.1 Các bên liên quan được xác định

- **Bác sĩ (Doctor/Physician):** Chịu trách nhiệm chẩn đoán, điều trị và quản lý bệnh nhân thông qua hệ thống.
- **Nhà khoa học dữ liệu (Data Scientist/ML Engineer):** Chịu trách nhiệm xây dựng, huấn luyện, triển khai và giám sát các mô hình học máy dự đoán đột quỵ.

1.2 Các trường hợp sử dụng (Use Cases) của Bác sĩ

1.2.1 Xác thực và kiểm soát truy cập

- **Trường hợp sử dụng:** Đăng nhập an toàn vào bảng điều khiển dành cho bác sĩ.
- **Luồng thực hiện:**
 1. Bác sĩ truy cập trang đăng nhập của hệ thống.
 2. Nhập tên người dùng và mật khẩu được cấp.
 3. Hệ thống xác thực thông tin và cấp quyền truy cập dựa trên vai trò vào bảng điều khiển bác sĩ.

1.2.2 Quản lý bệnh nhân

- **Trường hợp sử dụng:** Thêm bệnh nhân mới với dữ liệu lâm sàng.
- **Luồng thực hiện:**
 1. Truy cập biểu mẫu "Thêm bệnh nhân mới" trên bảng điều khiển.
 2. Nhập thông tin nhân khẩu học (tên, tuổi, giới tính) và các chỉ số lâm sàng (tăng huyết áp, bệnh tim, đường huyết, BMI, tình trạng hút thuốc, v.v.).
 3. Gửi dữ liệu. Hệ thống xác thực đầu vào và ngay lập tức dự đoán nguy cơ đột quỵ.
- **Trường hợp sử dụng:** Cập nhật dữ liệu bệnh nhân và tính toán lại nguy cơ.

- **Luồng thực hiện:**

1. Mở chi tiết bệnh nhân và nhấp nút "Chỉnh sửa".
2. Các trường thông tin trở nên có thể chỉnh sửa.
3. Sửa đổi dữ liệu cần thiết và nhấp "Lưu".
4. Hệ thống xác thực và tính toán lại dự đoán nguy cơ đột quỵ.

1.2.3 Đánh giá rủi ro và hỗ trợ quyết định lâm sàng

- **Trường hợp sử dụng:** Nhận đánh giá rủi ro do AI cung cấp cho bệnh nhân.

- **Luồng thực hiện:**

1. Khi dữ liệu bệnh nhân được gửi hoặc cập nhật, hệ thống xử lý các đặc điểm lâm sàng.
2. Nếu mô hình ML được triển khai, hệ thống dự đoán xác suất đột quỵ.
3. Nếu không, công cụ dựa trên luật sẽ tính điểm rủi ro theo các yếu tố có trọng số (ví dụ: tuổi > 60, tăng huyết áp).
4. Kết quả dự đoán được hiển thị nổi bật với huy hiệu mã hóa màu (Nguy cơ cao/Thấp).

1.3 Các trường hợp sử dụng (Use Cases) của Nhà khoa học dữ liệu

1.3.1 Thu thập và xuất dữ liệu

- **Trường hợp sử dụng:** Tải xuống bộ dữ liệu hoàn chỉnh ở định dạng JSON hoặc CSV.

- **Luồng thực hiện:**

1. Điều hướng đến mục "Thu thập & Xuất dữ liệu".
2. Nhấp nút "Xuất JSON" hoặc "Xuất CSV".
3. Hệ thống truy vấn tất cả hồ sơ bệnh nhân, chuyển đổi sang định dạng yêu cầu và trình duyệt tải tệp xuống.

1.3.2 Phân tích và đảm bảo chất lượng dữ liệu

- **Trường hợp sử dụng:** Tạo tổng quan toàn diện về tập dữ liệu.

- **Luồng thực hiện:**

1. Nhấp nút "Phân tích tập dữ liệu".
2. Hệ thống tính toán các thống kê: tổng số bản ghi, phân phối nguy cơ, thống kê tuổi/BMI/glucose, tỷ lệ các yếu tố rủi ro.
3. Hiển thị kết quả để đánh giá chất lượng dữ liệu và phát hiện sai lệch.

- **Trường hợp sử dụng:** Xác định các vấn đề về tính đầy đủ dữ liệu.

- **Luồng thực hiện:**

1. Nhấp "Kiểm tra giá trị bị thiếu".
2. Hệ thống quét tất cả trường để tìm các giá trị rỗng hoặc không xác định và hiển thị bảng kết quả, cho biết số lượng và tỷ lệ phần trăm thiếu cho mỗi thuộc tính.

1.3.3 Huấn luyện mô hình Machine Learning

- **Trường hợp sử dụng:** Thực thi quy trình huấn luyện mô hình đầu-cuối.

- **Luồng thực hiện:**

1. Chọn thuật toán (ví dụ: Random Forest, SVM), đặt kích thước tập kiểm tra và số lần kiểm định chéo.
2. Gửi yêu cầu huấn luyện.
3. Hệ thống thực hiện quy trình: tải dữ liệu, tiền xử lý (xử lý giá trị thiếu, mã hóa biến, chuẩn hóa đặc trưng), phân chia tập huấn luyện/kiểm tra, huấn luyện mô hình, đánh giá và lưu trữ mô hình cùng siêu dữ liệu.

1.3.4 Quản lý và triển khai mô hình

- **Trường hợp sử dụng:** Kích hoạt mô hình đã huấn luyện để dự đoán trong môi trường sản phẩm.
- **Luồng thực hiện:**
 1. Xem đánh giá hiệu suất của các mô hình trong danh sách quản lý.
 2. Nhấp nút "Triển khai" cho mô hình đã chọn.
 3. Hệ thống xác minh tệp, tải mô hình vào bộ nhớ và cập nhật trạng thái thành "đã triển khai". Mô hình này sẽ được dùng cho tất cả dự đoán nguy cơ đột quỵ.

2 Lý thuyết về thuật toán

Việc lựa chọn thuật toán phù hợp là bước quan trọng, quyết định hiệu suất và khả năng diễn giải của mô hình dự đoán. Dựa trên đặc thù dữ liệu y tế mất cân bằng nghiêm trọng (tỷ lệ đột quỵ 4.98%) và yêu cầu ứng dụng lâm sàng, hệ thống hỗ trợ nhiều thuật toán học máy với các đặc điểm và cơ chế hoạt động khác nhau. Phần này trình bày chi tiết lý thuyết toán học, nguyên lý hoạt động và đặc điểm của từng thuật toán, cùng với phân tích khả năng ứng dụng trong bài toán phát hiện đột quỵ.

2.1 Hồi quy Logistic (Logistic Regression)

Nền tảng toán học

Logistic Regression là mô hình phân loại nhị phân dựa trên hàm sigmoid (logistic function). Khác với hồi quy tuyến tính dự đoán giá trị liên tục, Logistic Regression dự đoán xác suất một mẫu thuộc vào lớp dương tính (đột quỵ).

Hàm quyết định: Mô hình tính toán tổ hợp tuyến tính của các đặc trưng đầu vào:

$$z = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{w}^T \mathbf{x} \quad (1)$$

trong đó $\mathbf{w}$ là vector trọng số (weights) và $\mathbf{x}$ là vector đặc trưng.

Hàm Sigmoid: Để chuyển đổi z thành xác suất trong khoảng $[0, 1]$, mô hình áp dụng hàm sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = P(y = 1 | \mathbf{x}) \quad (2)$$

Hàm sigmoid có các tính chất quan trọng:

- Tiệm cận về 0 khi $z \rightarrow -\infty$ và tiệm cận về 1 khi $z \rightarrow +\infty$.
- Đạo hàm đơn giản: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, thuận lợi cho tối ưu hóa.
- Đối xứng qua điểm $(0, 0.5)$.

Hàm mất mát (Loss Function): Logistic Regression sử dụng hàm Binary Cross-Entropy (Log Loss):

$$L(\mathbf{w}) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right] \quad (3)$$

trong đó m là số mẫu, $y^{(i)}$ là nhãn thực (0 hoặc 1), $\hat{y}^{(i)} = \sigma(\mathbf{w}^T \mathbf{x}^{(i)})$ là xác suất dự đoán.

Tối ưu hóa: Các trọng số $\mathbf{w}$ được học bằng gradient descent hoặc các biến thể (Adam, L-BFGS):

$$\mathbf{w} := \mathbf{w} - \alpha \nabla L(\mathbf{w}) \quad (4)$$

Xử lý mất cân bằng lớp

Trong dữ liệu y tế mất cân bằng, Logistic Regression hỗ trợ tham số `class_weight='balanced'` để điều chỉnh hàm mất mát:

$$w_{class} = \frac{n_{samples}}{n_{classes} \times n_{samples_class}} \quad (5)$$

Điều này tăng trọng số phạt cho lỗi phân loại sai lớp thiểu số (đột quỵ), khuyến khích mô hình học tốt hơn lớp này.

Ưu điểm

- *Tính diễn giải cao*: Hệ số w_i cho biết tác động của đặc trưng x_i lên log-odds. Ví dụ: $w_{age} > 0$ nghĩa là tuổi càng cao, nguy cơ đột quỵ càng tăng.
- *Hiệu quả tính toán*: Huấn luyện và dự đoán rất nhanh, thích hợp cho hệ thống thời gian thực.
- *Ổn định*: Ít nhạy cảm với nhiễu so với mô hình phi tuyến phức tạp.
- *Đầu ra xác suất*: Trực tiếp cung cấp xác suất, hữu ích cho quyết định lâm sàng.
- *Regularization tích hợp*: Hỗ trợ L1 (Lasso) và L2 (Ridge) để tránh overfitting.

Nhược điểm và hạn chế

- *Giả định tuyến tính*: Giả định mối quan hệ tuyến tính giữa log-odds và đặc trưng, không hiệu quả với quan hệ phi tuyến phức tạp.
- *Yêu cầu tiền xử lý*: Cần chuẩn hóa đặc trưng số và mã hóa biến phân loại.
- *Nhạy cảm với đa cộng tuyến*: Khi các đặc trưng tương quan cao, hệ số có thể không ổn định.

Phù hợp với bài toán đột quỵ

Logistic Regression là lựa chọn lý tưởng cho ứng dụng y tế vì:

- Bác sĩ có thể hiểu và giải thích quyết định của mô hình.
- Dự đoán nhanh, phù hợp với môi trường lâm sàng.
- Hỗ trợ tốt dữ liệu mất cân bằng với `class_weight`.
- Có thể điều chỉnh ngưỡng quyết định để ưu tiên Recall (giảm bỏ sót ca bệnh).

2.2 Rừng ngẫu nhiên (Random Forest)

Nguyên lý Ensemble Learning

Random Forest là thuật toán ensemble dựa trên Bootstrap Aggregating (Bagging), kết hợp nhiều cây quyết định độc lập để cải thiện hiệu suất và giảm overfitting.

Bootstrap Sampling: Với tập huấn luyện D có n mẫu, Random Forest tạo B tập con (bootstrap samples) bằng cách lấy mẫu có hoàn lại:

$$D_b = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad b = 1, 2, \dots, B \quad (6)$$

Mỗi tập D_b có kích thước n nhưng chứa khoảng 63.2% mẫu duy nhất từ D (các mẫu còn lại là "out-of-bag").

Feature Randomness: Tại mỗi nút phân chia của cây, thay vì xét tất cả p đặc trưng, Random Forest chỉ xét ngẫu nhiên $m \approx \sqrt{p}$ đặc trưng. Điều này:

- Giảm tương quan giữa các cây (decorrelation).
- Ngăn chặn các đặc trưng mạnh chiếm ưu thế.
- Tăng tính đa dạng của ensemble.

Dự đoán: Đối với phân loại, Random Forest dùng "bỏ phiếu đa số" (majority voting):

$$\hat{y} = \text{mode}\{h_b(\mathbf{x})\}_{b=1}^B \quad (7)$$

hoặc tính trung bình xác suất từ các cây:

$$P(y = 1|\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B P_b(y = 1|\mathbf{x}) \quad (8)$$

Cây quyết định (Decision Tree)

Mỗi cây trong Random Forest là một cây quyết định CART (Classification and Regression Tree):

- *Phân chia nút*: Tại mỗi nút, chọn đặc trưng j và ngưỡng t để tối đa hóa giảm độ tạp (impurity reduction). Tiêu chí Gini:

$$G = \sum_{k=1}^K p_k(1 - p_k) \quad (9)$$

với p_k là tỷ lệ mẫu thuộc lớp k tại nút.

- *Điều kiện dừng*: Cây phát triển đến khi đạt độ sâu tối đa, số mẫu tối thiểu ở nút lá, hoặc không thể phân chia thêm.

Feature Importance

Random Forest cung cấp độ quan trọng đặc trưng dựa trên "Mean Decrease Impurity"(MDI):

$$\text{Importance}(x_j) = \frac{1}{B} \sum_{b=1}^B \sum_{t \in T_b} \mathbb{1}_{v(t)=j} \cdot \Delta G(t) \quad (10)$$

trong đó $v(t)$ là đặc trưng được chọn tại nút t , $\Delta G(t)$ là giảm Gini impurity tại nút đó.

Ưu điểm

- *Xử lý phi tuyến*: Không yêu cầu giả định về quan hệ giữa đặc trưng và nhãn.
- *Mạnh mẽ với nhiễu*: Ensemble giảm phương sai, ít bị ảnh hưởng bởi nhiễu và outliers.
- *Ít cần tiền xử lý*: Không cần chuẩn hóa, tự động xử lý đa cộng tuyến.
- *Feature Importance*: Giúp hiểu đặc trưng nào quan trọng nhất.
- *Out-of-Bag Error*: Đánh giá không thiên lệch mà không cần tập validation riêng.

Nhược điểm

- *Khó diễn giải*: Kết hợp nhiều cây làm mô hình phức tạp, khó giải thích quyết định cụ thể.
- *Chi phí tính toán*: Huấn luyện và dự đoán chậm hơn mô hình tuyến tính.
- *Bộ nhớ*: Lưu trữ nhiều cây tốn bộ nhớ.
- *Thiên về lớp đa số*: Trong dữ liệu mất cân bằng nghiêm trọng, có thể dự đoán luôn lớp đa số nếu không điều chỉnh.

Xử lý dữ liệu mất cân bằng

Để cải thiện hiệu suất với dữ liệu đột quy mất cân bằng, có thể áp dụng:

- `class_weight='balanced'`: Tăng trọng số cho lớp thiểu số trong hàm mất mát.
- *Balanced Random Forest*: Undersample lớp đa số trong mỗi bootstrap.
- *SMOTE*: Oversampling tổng hợp cho lớp thiểu số.
- *Điều chỉnh ngưỡng*: Thay vì ngưỡng 0.5, dùng ngưỡng thấp hơn để tăng Recall.

2.3 Máy vector hỗ trợ (SVM)

Nền tảng toán học

SVM tìm siêu phẳng phân tách tối ưu giữa hai lớp bằng cách tối đa hóa lề (margin) - khoảng cách từ siêu phẳng đến điểm gần nhất của mỗi lớp.

Trường hợp tuyến tính phân tách: Siêu phẳng được định nghĩa bởi:

$$\mathbf{w}^T \mathbf{x} + b = 0 \quad (11)$$

với $\mathbf{w}$ là vector pháp tuyến và b là hệ số chặn.

Mục tiêu tối ưu hóa:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{sao cho} \quad y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1, \forall i \quad (12)$$

Lề được tính là $\frac{2}{\|\mathbf{w}\|}$, tối thiểu hóa $\|\mathbf{w}\|^2$ tương đương tối đa hóa lề.

Soft Margin: Trong thực tế, dữ liệu không tuyến tính phân tách hoàn toàn. SVM dùng "soft margin" với biến slack ξ_i :

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \quad (13)$$

$$\text{sao cho} \quad y^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (14)$$

Tham số C kiểm soát trade-off giữa tối đa hóa lề và giảm lỗi phân loại:

- C lớn: phạt nặng lỗi, lề nhỏ hơn, có thể overfit.
- C nhỏ: cho phép nhiều lỗi, lề rộng hơn, tổng quát hóa tốt hơn.

Kernel Trick

Để xử lý dữ liệu phi tuyến, SVM dùng "kernel trick" ánh xạ dữ liệu sang không gian chiều cao hơn mà không cần tính toán tường minh.

Kernel RBF (Radial Basis Function/Gaussian):

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2) \quad (15)$$

Tham số γ kiểm soát độ ảnh hưởng của một điểm:

- γ lớn: ảnh hưởng cục bộ, biên phức tạp, dễ overfit.
- γ nhỏ: ảnh hưởng toàn cục, biên đơn giản hơn.

Các kernel khác:

- *Linear*: $K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$.
- *Polynomial*: $K(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x}^T \mathbf{x}' + r)^d$.
- *Sigmoid*: $K(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x}^T \mathbf{x}' + r)$.

Dự đoán xác suất

SVM ban đầu không cung cấp xác suất mà chỉ quyết định nhị phân. Khi bật `probability=True`, sử dụng Platt Scaling:

$$P(y = 1 | \mathbf{x}) = \frac{1}{1 + \exp(Af(\mathbf{x}) + B)} \quad (16)$$

với $f(\mathbf{x})$ là khoảng cách đến siêu phẳng, A và B học bằng cross-validation.

Ưu điểm

- *Hiệu quả trong không gian chiều cao*: Hoạt động tốt khi số đặc trưng lớn hơn số mẫu.
- *Mạnh mẽ với outliers*: Chỉ phụ thuộc vào support vectors (điểm gần biên).
- *Linh hoạt*: Kernel cho phép học biên phức tạp.
- *Nền tảng lý thuyết vững*: Dựa trên lý thuyết tối ưu lỗi và margin theory.

Nhược điểm

- *Chi phí tính toán*: Huấn luyện $O(m^2)$ đến $O(m^3)$ với m mẫu, chậm với tập lớn.
- *Nhạy cảm siêu tham số*: Cần điều chỉnh C và γ cẩn thận.
- *Khó diễn giải*: Không cung cấp thông tin về tầm quan trọng đặc trưng.
- *Xác suất không hiệu quả*: Platt Scaling tăng chi phí huấn luyện và có thể không chính xác.

Phù hợp với dữ liệu mất cân bằng

- Tham số `class_weight='balanced'` điều chỉnh C cho từng lớp:

$$C_{class} = C \times w_{class} \quad (17)$$

- Kernel RBF có thể học biên phức tạp để phân tách lớp thiểu số.
- Hiệu quả với tập dữ liệu vừa phải như bài toán đột quỵ (~ 5000 mẫu).

2.4 Naive Bayes (GaussianNB)

Nền tảng xác suất

Naive Bayes dựa trên định lý Bayes với giả định "naive" (ngây thơ) rằng các đặc trưng độc lập có điều kiện với nhau khi biết nhãn.

Định lý Bayes:

$$P(y|\mathbf{x}) = \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})} \quad (18)$$

Giả định Naive: Các đặc trưng độc lập có điều kiện:

$$P(\mathbf{x}|y) = P(x_1, x_2, \dots, x_p|y) = \prod_{j=1}^p P(x_j|y) \quad (19)$$

Do đó:

$$P(y|\mathbf{x}) \propto P(y) \prod_{j=1}^p P(x_j|y) \quad (20)$$

Dự đoán: chọn lớp có xác suất hậu nghiệm cao nhất:

$$\hat{y} = \arg \max_y P(y) \prod_{j=1}^p P(x_j|y) \quad (21)$$

Gaussian Naive Bayes

Giả định đặc trưng liên tục x_j tuân theo phân phối chuẩn trong mỗi lớp:

$$P(x_j|y=k) = \frac{1}{\sqrt{2\pi\sigma_{jk}^2}} \exp\left(-\frac{(x_j - \mu_{jk})^2}{2\sigma_{jk}^2}\right) \quad (22)$$

với μ_{jk} và σ_{jk}^2 là trung bình và phương sai của x_j trong lớp k , ước lượng từ dữ liệu huấn luyện.

Ước lượng tham số:

$$\mu_{jk} = \frac{1}{n_k} \sum_{i:y^{(i)}=k} x_j^{(i)}, \quad \sigma_{jk}^2 = \frac{1}{n_k} \sum_{i:y^{(i)}=k} (x_j^{(i)} - \mu_{jk})^2 \quad (23)$$

$$P(y=k) = \frac{n_k}{m} \quad (24)$$

Ưu điểm

- *Tốc độ cực nhanh:* Huấn luyện chỉ cần tính mean và variance, dự đoán là phép nhân xác suất.
- *Hiệu quả bộ nhớ:* Chỉ lưu μ_{jk} , σ_{jk}^2 và $P(y=k)$.
- *Ít tham số:* Không cần điều chỉnh siêu tham số.
- *Hoạt động tốt với ít dữ liệu:* Ước lượng tham số đơn giản ít cần dữ liệu nhiều.
- *Xác suất trực tiếp:* Đầu ra là xác suất, hữu ích cho quyết định lâm sàng.

Nhược điểm

- *Giả định độc lập không thực tế:* Trong dữ liệu y tế, các đặc trưng thường tương quan (ví dụ: tuổi, tăng huyết áp, bệnh tim).
- *Giả định Gaussian không phù hợp:* Một số đặc trưng không tuân theo phân phối chuẩn.
- *Zero probability:* Nếu một giá trị chưa xuất hiện trong huấn luyện, $P(x_j|y) = 0$ làm toàn bộ tích bằng 0 (cần Laplace smoothing).
- *Không xử lý phi tuyến:* Biên quyết định là tuyến tính hoặc bậc hai trong không gian log-probability.

Phù hợp với bài toán đột quỵ

- Rất nhanh, phù hợp hệ thống cần phản hồi thời gian thực.
- Hoạt động như baseline hoặc mô hình dự phòng.
- Có thể bổ sung Laplace smoothing để xử lý giá trị hiếm.
- Cần đánh giá liệu giả định độc lập có ảnh hưởng lớn đến hiệu suất hay không.

2.5 So sánh tổng quan các thuật toán

2.6 Chiến lược lựa chọn thuật toán

Dựa trên đặc thù bài toán phát hiện đột quỵ, các tiêu chí lựa chọn thuật toán:

- **Ưu tiên Recall:** Trong y tế, bỏ sót ca đột quỵ (false negative) nguy hiểm hơn cảnh báo sai (false positive). Thuật toán cần tối ưu hóa Recall.
- **Tính diễn giải:** Bác sĩ cần hiểu lý do dự đoán để đưa ra quyết định lâm sàng. Logistic Regression và Naive Bayes có lợi thế.
- **Xử lý mất cân bằng:** Tỷ lệ 4.98% ca đột quỵ yêu cầu thuật toán hỗ trợ tốt dữ liệu mất cân bằng (class weights, SMOTE).

Tiêu chí	LR	RF	SVM	NB
Tốc độ huấn luyện	Nhanh	Trung bình	Chậm	Rất nhanh
Tốc độ dự đoán	Nhanh	Trung bình	Trung bình	Rất nhanh
Diễn giải	Cao	Trung bình	Thấp	Cao
Xử lý phi tuyến	Thấp	Cao	Cao	Thấp
Xử lý mất cân bằng	Tốt	Trung bình	Tốt	Trung bình
Yêu cầu tiền xử lý	Cao	Thấp	Cao	Trung bình
Nhạy cảm siêu tham số	Thấp	Trung bình	Cao	Rất thấp

Bảng 1: So sánh đặc điểm các thuật toán

- **Hiệu quả tính toán:** Hệ thống lâm sàng cần dự đoán nhanh. Logistic Regression và Naive Bayes phù hợp nhất.
- **Độ chính xác:** Cân bằng giữa hiệu suất và các tiêu chí trên. Random Forest thường có accuracy cao nhưng cần đánh giá Recall.

Khuyến nghị sơ bộ (trước khi đánh giá thực nghiệm):

1. *Baseline:* Logistic Regression với `class_weight='balanced'` - đơn giản, nhanh, dễ diễn giải.
2. *Mô hình phức tạp:* Random Forest và SVM với điều chỉnh class weight.
3. *Dự phòng:* Naive Bayes cho hệ thống cần phản hồi thời gian thực.

3 Kiến trúc hệ thống

3.1 Thành phần Backend

- **Web Framework (Flask):** Lõi backend xây dựng bằng Flask, micro-framework Python, gọn nhẹ, linh hoạt, hệ sinh thái mạnh.
- **Routing và logic nghiệp vụ (app.py):** app.py là "bộ não" của ứng dụng. Định nghĩa các route như /login, /doctor/dashboard, /data_scientist/dashboard và các API. Tại đây xử lý HTTP, xác thực, phân quyền, CRUD bệnh nhân và tương tác cơ sở dữ liệu.
- **Cơ sở dữ liệu và ORM (models.py):** Sử dụng SQLAlchemy ORM để làm việc với cơ sở dữ liệu.
 - *Mô hình dữ liệu:* models.py định nghĩa các lớp Python (User, Patient) ánh xạ bảng CSDL, cho phép thao tác qua đối tượng thay vì SQL thô.
 - *Cơ sở dữ liệu:* Mặc định dùng SQLite (hospital.db) để dễ cài đặt và chạy thử; có thể chuyển sang PostgreSQL qua biến môi trường.
- **Cấu hình (config.py):** Quản lý cấu hình cho các môi trường (phát triển, kiểm thử, sản phẩm). Thông tin nhạy cảm như chuỗi kết nối và secret key được quản lý tại đây, tách biệt mã nguồn và cấu hình.

3.2 Thành phần Frontend

- **Công nghệ:** Giao diện xây dựng bằng HTML, CSS và JavaScript thuần.
- **Engine mẫu (Jinja2):** Các tệp HTML trong templates được xử lý bởi Jinja2 để hiển thị dữ liệu động và dùng cấu trúc logic như vòng lặp, điều kiện.
- **Giao diện người dùng (templates/ và static/):**
 - *Bảng điều khiển bác sĩ (doctor_dashboard.html):* Quản lý bệnh nhân (thêm, xem, sửa), xem thống kê và kết quả dự đoán; hỗ trợ panel chi tiết bệnh nhân và chỉnh sửa tại chỗ bằng JavaScript.
 - *Bảng điều khiển nhà khoa học dữ liệu (data_scientist_dashboard.html):* Cung cấp công cụ phân tích, huấn luyện và quản lý mô hình.
 - *Tài sản tĩnh:* Tệp CSS (style.css) và JavaScript trong tương lai nằm trong static và được phục vụ trực tiếp.

3.3 Quy trình học máy

Quy trình học máy được thiết kế khép kín trong bảng điều khiển nhà khoa học dữ liệu, gồm các bước:

- **Bước 1: Chuẩn bị dữ liệu (Data Preparation)**

- *Xuất dữ liệu*: Xuất toàn bộ dữ liệu bệnh nhân ra CSV hoặc JSON làm nguồn dữ liệu cho huấn luyện.
- *Tải và tiền xử lý*: Khi bắt đầu huấn luyện, hệ thống tự động tải dữ liệu và áp dụng:
 - * Xử lý giá trị thiếu (ví dụ: điền giá trị trung bình cho cột BMI).
 - * Mã hóa biến phân loại (ví dụ: One-Hot Encoding cho `gender`, `work_type`, `smoking_status`).
 - * Chuẩn hóa đặc trưng số (StandardScaler cho `age`, `avg_glucose_level`, `bmi`).

- **Bước 2: Huấn luyện và đánh giá (Training & Evaluation)**

- *Lựa chọn cấu hình*: Chọn thuật toán (Random Forest, SVM), tỷ lệ tập kiểm tra và số fold kiểm định chéo.
- *Phân chia dữ liệu*: Chia dữ liệu đã tiền xử lý thành tập huấn luyện và kiểm tra.
- *Huấn luyện mô hình*: Huấn luyện trên tập huấn luyện; nếu có, thực hiện kiểm định chéo để đánh giá ổn định.
- *Đánh giá hiệu suất*: Tính Accuracy, Precision, Recall, F1 và ma trận nhầm lẫn.

- **Bước 3: Lưu trữ và quản lý (Model Storage & Management)**

- *Lưu trữ mô hình*: Lưu mô hình (đối tượng Python) và siêu dữ liệu (thuật toán, chỉ số, tham số, ngày tạo) vào tệp (ví dụ: `.pkl`) và ghi nhận trong CSDL.
- *Hiển thị danh sách*: Giao diện "Quản lý mô hình" hiển thị danh sách các mô hình đã huấn luyện để so sánh.

- **Bước 4: Triển khai (Deployment)**

- *Kích hoạt mô hình*: Chọn mô hình có hiệu suất tốt nhất và nhấn "Triển khai".
- *Tải và sử dụng*: Hệ thống tải tệp mô hình vào bộ nhớ và đặt làm mô hình hoạt động; từ thời điểm này, mọi dự đoán nguy cơ đột quỵ sẽ dùng mô hình học máy thay vì công cụ dựa trên luật.

4 Tập dữ liệu và tiền xử lý

4.1 Mô tả tập dữ liệu

Tổng quan

Dựa trên notebook `report/train_model.ipynb` (mục 3: Phân tích dữ liệu), tập dữ liệu được nạp từ tệp `brain_stroke.csv` với **4,981** bản ghi và **11** cột. Trước khi phân tích, notebook chuẩn hoá tên cột theo `rename_map = {Residence_type -> residence_type, Avg_glucose_level -> avg_glucose_level}` để thống nhất schema.

Danh sách cột: `gender`, `age`, `hypertension`, `heart_disease`, `ever_married`, `work_type`, `residence_type`, `avg_glucose_level`, `bmi`, `smoking_status`, `stroke`.

Kiểu dữ liệu: `age`, `avg_glucose_level`, `bmi` (`float64`); `hypertension`, `heart_disease`, `stroke` (`int64`); các cột còn lại (`object`).

Thiếu dữ liệu: Bảng kiểm tra cho thấy không có giá trị thiếu.

Thống kê số liệu (describe cho các cột số):

- `age`: count=4,981; mean=43.4199; std=22.6628; min=0.08; 25%=25; 50%=45; 75%=61; max=82.
- `avg_glucose_level`: count=4,981; mean=105.9436; std=45.0754; min=55.12; 25%=77.23; 50%=91.85; 75%=113.86; max=271.74.
- `bmi`: count=4,981; mean=28.4982; std=6.7905; min=14.0; 25%=23.7; 50%=28.1; 75%=32.6; max=48.9.

Thống kê phân loại: gender (mode=Female, freq=2,907), ever_married (Yes, 3,280), work_type (Private, 2,860), residence_type (Urban, 2,532), smoking_status (never smoked, 1,838).

```
Loading CSV from: c:\Users\Lenovo\Desktop\Master\Intelligence_Sys\stroke-warning-system\brain_stroke.csv

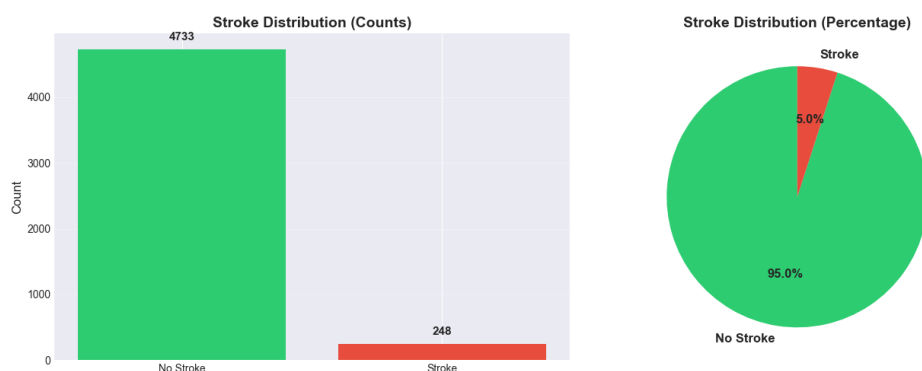
Dataset shape: (4981, 11)

Class distribution:
stroke
0      4733
1       248
Name: count, dtype: int64
```

Hình 2: Một số bản ghi đầu tiên từ tập dữ liệu

Phân phối của nhãn

stroke=1 có 248 mẫu (~4.98%); stroke=0 có 4,733 mẫu (~95.02%); tỉ lệ mất cân bằng xấp xỉ 1 : 19.1. Biểu đồ cột và tròn trong notebook cho thấy mất cân bằng mạnh, thông báo “*Imbalance detected - Stroke cases are only 4.98% of the data*”.

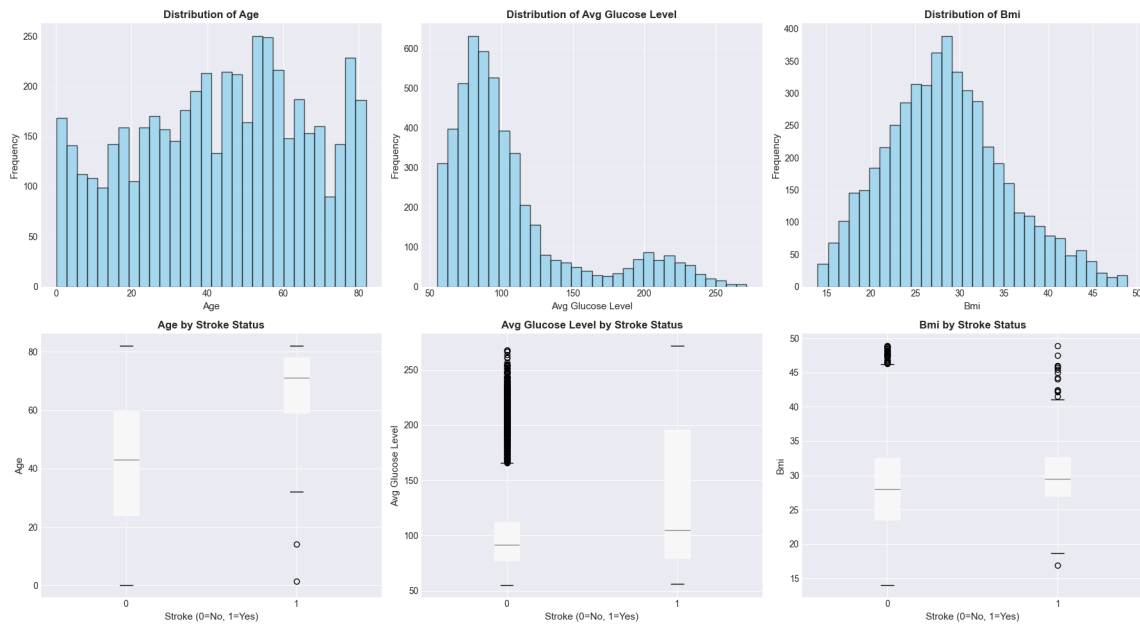


Hình 3: Phân phối nhãn Stroke (đếm và phần trăm)

Phân phối của những features khác

Phân tích các đặc trưng số (age, avg_glucose_level, bmi) bằng histogram và boxplot theo nhãn stroke:

- age: phân phối rộng, trung vị khoảng 45; nhóm có stroke=1 nghiêng về tuổi cao hơn (boxplot).
- avg_glucose_level: có đuôi phải (right-skew), nhóm stroke=1 thường có mức đường huyết cao hơn.
- bmi: tập trung quanh 28-32, chênh lệch vừa phải theo nhãn.



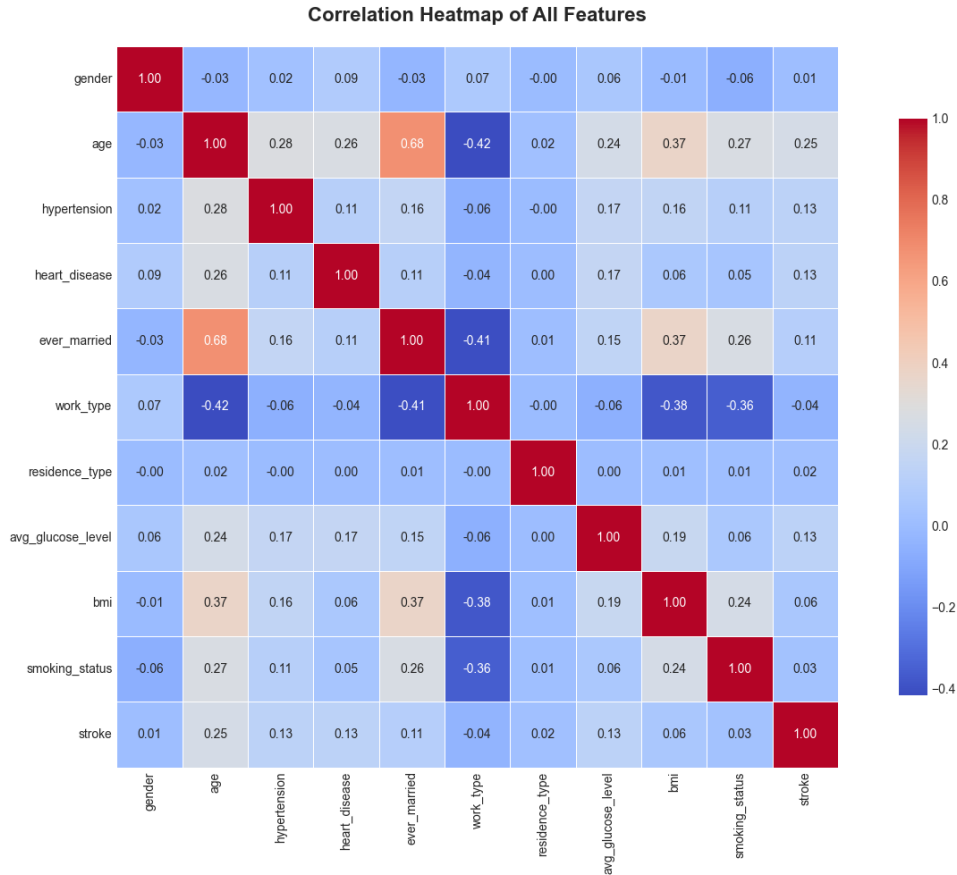
Hình 4: Histogram và Boxplot theo nhân cho các thuộc tính số

Sự tương quan

Ma trận tương quan cho các đặc trưng số cho thấy:

- **age** tương quan dương với nhân **stroke**.
- **avg_glucose_level** tương quan dương mức vừa với **stroke** và yếu với **bmi**.
- **bmi** tương quan yếu với **stroke**.

Nhận xét này phù hợp với trực giác y khoa: tuổi và đường huyết là yếu tố rủi ro nổi bật.

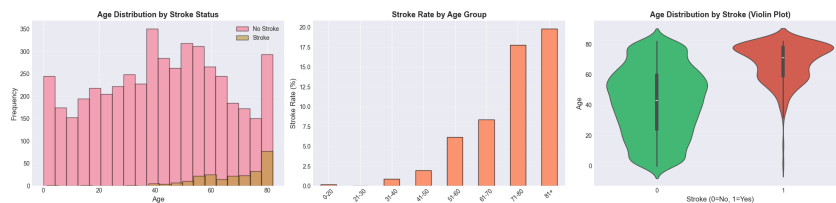


Hình 5: Ma trận tương quan giữa các đặc trưng số

Phân tích feature age – Feature chính

age nổi bật trong *feature importance* (Random Forest) và biểu đồ hộp theo nhãn:

- Nhóm **stroke=1** có phân vị 25/50/75 cao hơn đáng kể so với nhóm **stroke=0**.
- Khi phân nhóm theo các ngưỡng (ví dụ: < 30, 30–50, > 50), tỷ lệ đột quỵ tăng rõ rệt ở nhóm tuổi cao.
- Trong mô hình, **age** thường là thuộc tính quan trọng nhất, phản ánh tác động mạnh đến nguy cơ.



Hình 6: Phân tích chi tiết thuộc tính **age** theo nhãn

Kết luận: Tập dữ liệu mất cân bằng nghiêm trọng, không có giá trị thiếu, và gồm cả đặc trưng số lẫn phân loại. Các báo cáo đánh giá cần ưu tiên *Recall*, *ROC-AUC*, đồng thời cân nhắc *class_weight*/điều chỉnh ngưỡng và kỹ thuật lấy mẫu để giảm bỏ sót (FN).

5 Quy trình huấn luyện và triển khai model

Hệ thống cung cấp quy trình học máy toàn diện, tích hợp trực tiếp vào giao diện web, hợp lý hóa vòng đời mô hình từ thu thập dữ liệu đến triển khai và giám sát.

5.1 Các bước trong quy trình

5.1.1 Bước 1: Thu thập và chuẩn bị dữ liệu

- **Thu thập dữ liệu:** Hai nguồn chính:
 - *Nhập liệu thủ công:* Bác sĩ thêm bệnh nhân mới qua giao diện web với đầy đủ thuộc tính.
 - *Nhập CSV:* Sử dụng script `migrate_data.py` để nhập dữ liệu từ `brain_stroke.csv` vào CSDL; script có cơ chế tránh trùng lặp (dừng nếu đã có hơn 10 bệnh nhân).
- **Lưu trữ tập trung:** Dữ liệu lưu trong SQLite (hoặc PostgreSQL ở sản phẩm) qua bảng `Patient` để đảm bảo nhất quán và dễ truy xuất.
- **Kiểm tra chất lượng dữ liệu:** Trước khi huấn luyện, hệ thống kiểm tra:
 - Đủ dữ liệu (tối thiểu 50 bản ghi) để huấn luyện đáng tin cậy.
 - Có cả trường hợp dương tính và âm tính để tránh thiên lệch.
 - Lọc bản ghi có `stroke_prediction = NULL` (chưa gán nhãn).
- **Tiền xử lý tự động:** Hàm `preprocess_data_for_training()` trong `app.py` thực hiện:
 - *Xử lý giá trị thiếu:* Điền giá trị thiếu ở cột số bằng trung vị (median).
 - *Mã hóa biến phân loại:* Dùng `LabelEncoder` cho `gender`, `ever_married`, `work_type`, `residence_type`, `smoking_status`.
 - *Chuẩn hóa đặc trưng:* Áp dụng `StandardScaler` cho các thuộc tính số.
 - *Phân tách X và y:* Tách X (không gồm `stroke`) và y (`stroke`).

5.1.2 Bước 2: Cấu hình và huấn luyện mô hình

- **Giao diện cấu hình:** Cho phép tùy chỉnh tham số huấn luyện:
 - *Lựa chọn thuật toán:* Hỗ trợ 5 thuật toán trong `get_model_by_algorithm()`:
 - * **Random Forest:** 200 cây, không giới hạn độ sâu, `class_weight='balanced'`.
 - * **Support Vector Machine (SVM):** Kernel RBF, hỗ trợ `predict_proba`, `class_weight='balanced'`.
 - * **Logistic Regression:** Max iterations=1000, `class_weight='balanced'`.
 - * **Naive Bayes:** GaussianNB.
 - *Tỷ lệ phân chia:* Chọn phần trăm dữ liệu cho tập kiểm tra (mặc định 20%).
 - *Kiểm định chéo:* Chọn số fold cho k-fold cross-validation (mặc định 5).
- **Phân chia dữ liệu có phân tầng:** Dùng `train_test_split` với `stratify=y` để duy trì tỷ lệ lớp.
- **Huấn luyện mô hình:** Route `/data_scientist/train_model` thực hiện:
 1. Khởi tạo mô hình đã chọn với siêu tham số tối ưu sẵn.
 2. Gọi `model.fit(X_train, y_train)` để huấn luyện.
 3. Huấn luyện đồng bộ, gửi thông báo trạng thái về giao diện.

5.1.3 Bước 3: Đánh giá hiệu suất

- **Dự đoán trên tập kiểm tra:** Dự đoán trên `X_test` mà mô hình chưa thấy.
- **Tính các chỉ số:** Tự động tính:
 - **Accuracy:** $\frac{TP+TN}{TP+TN+FP+FN}$.
 - **Precision:** $\frac{TP}{TP+FP}$.
 - **Recall:** $\frac{TP}{TP+FN}$.
 - **F1-Score:** $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$.
 - **Ma trận nhầm lẫn:** TN, FP, FN, TP.
 - **ROC-AUC:** Diện tích dưới đường cong ROC (áp dụng cho mô hình hỗ trợ `predict_proba`).
- **Kiểm định chéo:** Đánh giá độ ổn định bằng k-fold:
 - Chia tập huấn luyện thành k phần.
 - Mỗi lượt: huấn luyện trên k-1 phần, đánh giá trên phần còn lại.
 - Tính mean và std điểm số.
 - Độ lệch chuẩn thấp cho thấy mô hình ổn định.
- **Độ quan trọng thuộc tính:** Với mô hình hỗ trợ (Random Forest), tính và sắp xếp 10 thuộc tính quan trọng nhất.

5.1.4 Bước 4: Lưu trữ và quản lý phiên bản

- **Lưu trữ mô hình:** Dùng pickle:
 - *Tệp mô hình:* Lưu trong `model/` tên `<algorithm>_<timestamp>_model.pkl` (ví dụ: `random_forest_20240315_123456.pkl`).
 - *Nội dung:* Bao gồm `model`, `scaler`, `feature_names` để đảm bảo sắp xếp đặc trưng đúng.
- **Lịch sử mô hình:** Lưu siêu dữ liệu vào `model/models_history.json`:
 - Mỗi mục: thuật toán, kích thước dữ liệu, chỉ số, thời gian, người huấn luyện, tên tệp, trạng thái triển khai.
 - Giữ lại 20 mô hình gần nhất để tránh lưu trữ quá nhiều.
- **Giao diện quản lý mô hình:** Route `/data_scientist/list_models` cung cấp API hiển thị danh sách mô hình đã huấn luyện.

5.1.5 Bước 5: Triển khai mô hình

- **Lựa chọn mô hình:** Từ giao diện "Quản lý mô hình", nhà khoa học dữ liệu chọn mô hình tốt nhất để triển khai.
- **Quá trình triển khai:** Khi nhấn "Deploy", route `/data_scientist/deploy_model` thực hiện:
 1. Xác thực tệp mô hình tồn tại và có thể tải.
 2. Tải mô hình vào bộ nhớ bằng `pickle.load()`.
 3. Cập nhật `deployed=true` cho mô hình chọn trong `models_history.json`.
 4. Đặt các mô hình khác `deployed=false`.
 5. Lưu siêu dữ liệu mô hình đang hoạt động vào `model/metrics.json`.
 6. Gán mô hình vào biến toàn cục `TRAINED_MODEL` để kích hoạt.
- **Tích hợp dự đoán:** Sau triển khai, hàm `predict_stroke()` trong `app.py`:
 - Kiểm tra `TRAINED_MODEL` khác `None`.
 - Nếu có, gọi `predict_with_ml_model()` để dự đoán bằng mô hình học máy.
 - Nếu không, dùng hệ thống dựa trên luật làm dự phòng.

- **Dự đoán với ML:** Hàm `predict_with_ml_model()`:

1. Tạo DataFrame từ dữ liệu bệnh nhân.
2. Mã hóa biến phân loại bằng LabelEncoder (nên lưu encoder trong sản phẩm).
3. Chọn và sắp xếp đặc trưng theo đúng thứ tự huấn luyện.
4. Áp dụng `scaler.transform()` để chuẩn hóa.
5. Gọi `model.predict()` để nhận dự đoán.
6. Chuyển kết quả (0/1) thành nhãn văn bản ('Low Risk'/'High Risk').

5.1.6 Bước 6: Giám sát và quản lý

- **Theo dõi mô hình hoạt động:** Giao diện hiển thị thuật toán, độ chính xác và thời gian triển khai.
- **Xóa mô hình không cần thiết:** Route `/data_scientist/delete_model` cho phép xóa mô hình cũ với cơ chế bảo vệ không xóa mô hình đang triển khai.
- **Huấn luyện lại định kỳ:** Khi có dữ liệu mới, có thể:
 - Huấn luyện mô hình mới với dữ liệu đầy đủ hơn.
 - So sánh hiệu suất với mô hình hiện tại.
 - Triển khai mô hình mới nếu hoạt động tốt hơn.
- **Cơ chế dự phòng:** Nếu mô hình ML lỗi trong dự đoán (ví dụ: dữ liệu đầu vào không hợp lệ), hệ thống tự động quay lại dự đoán dựa trên luật.

5.2 Ưu điểm của quy trình

- **Tích hợp liền mạch:** Quy trình từ chuẩn bị dữ liệu đến triển khai trong một giao diện web.
- **Linh hoạt:** Hỗ trợ nhiều thuật toán và tùy chỉnh siêu tham số.
- **Minh bạch:** Chỉ số đánh giá chi tiết giúp hiểu rõ hiệu suất.
- **Quản lý phiên bản:** Lưu lịch sử mô hình giúp so sánh và quay lại phiên bản trước.
- **An toàn:** Cơ chế dự phòng đảm bảo hệ thống luôn dự đoán được.

6 Kết quả đánh giá model

Sau quy trình huấn luyện và triển khai, phần này trình bày kết quả đánh giá tổng hợp cho bốn mô hình đã so sánh (Random Forest, SVM, Naive Bayes, Logistic Regression). Nội dung tóm tắt dựa trên thực thi trong notebook `train_model.ipynb` (mã huấn luyện nhiều mô hình), kết hợp phân tích về độ ổn định (cross-validation) và trực quan hoá (ROC, confusion matrix).

6.1 Các chỉ số đánh giá

Hệ thống dùng tập hợp chỉ số đánh giá toàn diện, mỗi chỉ số cung cấp một góc nhìn về hiệu suất mô hình:

- **Độ chính xác (Accuracy):**
 - Công thức: $\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$.
 - Ý nghĩa: Tỷ lệ dự đoán đúng tổng thể trên tập kiểm tra.
 - Hạn chế: Dễ gây hiểu lầm khi dữ liệu mất cân bằng lớp (ví dụ: tỷ lệ 4.98% ca đột quỵ).
- **Độ chuẩn xác (Precision):**
 - Công thức: $\text{Precision} = \frac{TP}{TP+FP}$.

- Ý nghĩa: Trong số ca được dự đoán là đột quỵ, có bao nhiêu ca thực sự là đột quỵ; quan trọng để giảm cảnh báo sai (false positives).

- **Độ nhạy (Recall/Sensitivity):**

- Công thức: $\text{Recall} = \frac{TP}{TP+FN}$.
- Ý nghĩa: Trong số ca đột quỵ thực tế, mô hình phát hiện được bao nhiêu; rất quan trọng vì bỏ sót (false negatives) có thể gây hậu quả nghiêm trọng.
- Mục tiêu: Thường ưu tiên Recall cao trong ứng dụng y tế.

- **Điểm F1 (F1-Score):**

- Công thức: $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$.
- Ý nghĩa: Trung bình điều hòa giữa Precision và Recall, cân bằng giảm false positives và false negatives.

- **Ma trận nhầm lẫn (Confusion Matrix):**

- Cấu trúc: Bảng 2x2 hiển thị TP, TN, FP, FN.
- Ý nghĩa: Cho biết các loại lỗi mô hình mắc phải.

- **ROC-AUC:**

- Ý nghĩa: Diện tích dưới đường cong ROC, đo khả năng phân biệt hai lớp, độc lập ngưỡng.
- Giá trị: Càng gần 1.0 càng tốt; 0.5 là ngẫu nhiên.

- **Kiểm định chéo (Cross-Validation):**

- Phương pháp: k-fold (thường k=5), huấn luyện trên k-1 phần và kiểm tra trên phần còn lại, lặp k lần.
- Ý nghĩa: Đánh giá ổn định và khả năng tổng quát hóa; std thấp cho thấy mô hình ít bị ảnh hưởng bởi phân chia dữ liệu.
- Kết quả: Mean và std điểm accuracy giữa các fold.

6.2 Kết quả huấn luyện các mô hình

Dựa trên notebook `train_model.ipynb`, bốn mô hình đã được huấn luyện và đánh giá trên cùng một tập dữ liệu. Kết quả chi tiết như sau:

6.2.1 Random Forest

Random Forest (200 cây, `class_weight='balanced'`) đạt các chỉ số sau trên tập kiểm tra:

Chỉ số	Giá trị
Accuracy	0.9478
Precision (Stroke)	0.0000
Recall (Stroke)	0.00
F1-Score (Stroke)	0.0000
ROC-AUC	0.8159
Cross-Val Mean	0.9496

Bảng 2: Kết quả hiệu suất của mô hình Random Forest

Phân tích: Random Forest đạt Accuracy rất cao (94.78%) và Cross-validation ổn định (94.96%). Tuy nhiên, Precision, Recall và F1-Score đều bằng 0, cho thấy mô hình **không phát hiện được bất kỳ ca đột quỵ nào** trên tập kiểm tra. ROC-AUC vẫn ở mức khá (0.8159), chứng tỏ mô hình có khả năng phân biệt nhưng cần điều chỉnh ngưỡng quyết định. Đây là dấu hiệu điển hình của việc mô hình thiên về dự đoán lớp đa số do mất cân bằng dữ liệu nghiêm trọng.

Chỉ số	Giá trị
Accuracy	0.7613
Precision (Stroke)	0.1299
Recall (Stroke)	0.66
F1-Score (Stroke)	0.2171
ROC-AUC	0.7824
Cross-Val Mean	0.7653

Bảng 3: Kết quả hiệu suất của mô hình SVM

6.2.2 Support Vector Machine (SVM)

SVM với kernel RBF và `class_weight='balanced'`:

Phân tích: SVM cho kết quả cân bằng hơn với Recall cao nhất (66%), phát hiện được 2/3 số ca đột quỵ. Accuracy (76.13%) thấp hơn Random Forest do không thiên về lớp đa số. Precision thấp (12.99%) nghĩa là có nhiều cảnh báo sai. ROC-AUC đạt 0.7824. Cross-validation ổn định (0.7653). SVM là mô hình **cân bằng tốt nhất** giữa Precision và Recall trong trường hợp này.

6.2.3 Naive Bayes

GaussianNB với giả định phân phối chuẩn:

Chỉ số	Giá trị
Accuracy	0.8606
Precision (Stroke)	0.1382
Recall (Stroke)	0.34
F1-Score (Stroke)	0.1965
ROC-AUC	0.8014
Cross-Val Mean	0.8572

Bảng 4: Kết quả hiệu suất của mô hình Naive Bayes

Phân tích: Naive Bayes đạt Accuracy cao (86.06%) và ROC-AUC tốt (0.8014). Recall trung bình (34%), chỉ phát hiện được 1/3 số ca đột quỵ. Precision thấp (13.82%). Cross-validation ổn định (85.72%). Mô hình này **thiên về an toàn** nhưng bỏ sót nhiều ca đột quỵ. Ưu điểm là tốc độ huấn luyện rất nhanh và ít tốn tài nguyên, phù hợp cho hệ thống cần phản hồi nhanh.

6.2.4 Logistic Regression

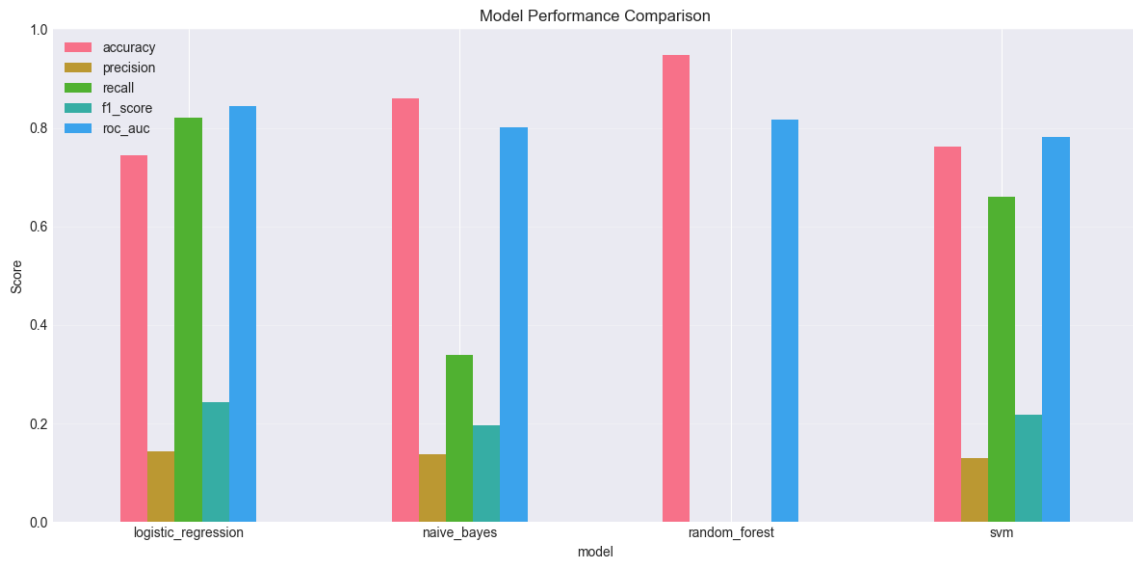
Logistic Regression với `class_weight='balanced'`:

Chỉ số	Giá trị
Accuracy	0.7442
Precision (Stroke)	0.1429
Recall (Stroke)	0.82
F1-Score (Stroke)	0.2433
ROC-AUC	0.8438
Cross-Val Mean	0.7400

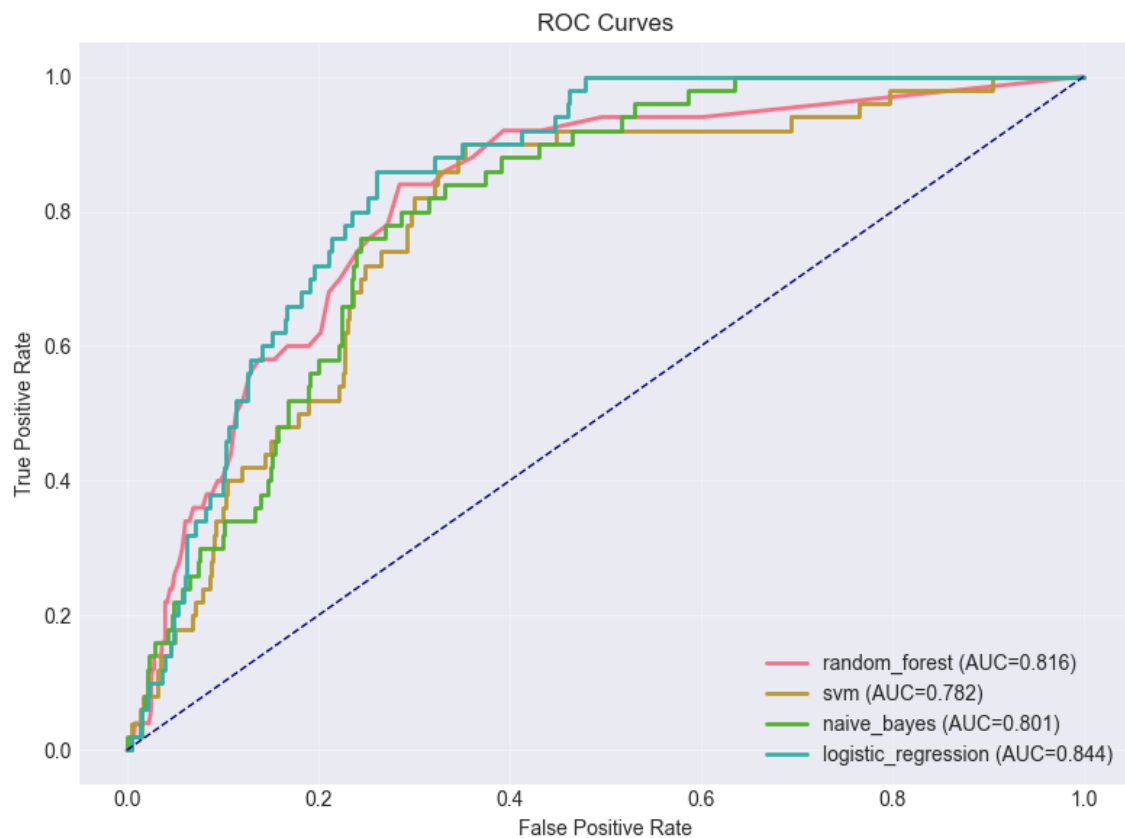
Bảng 5: Kết quả hiệu suất của mô hình Logistic Regression

Phân tích: Logistic Regression đạt **Recall cao nhất** (82%) và ROC-AUC cao nhất (0.8438). Mô hình này phát hiện được phần lớn các ca đột quỵ, ưu tiên giảm false negatives - điều quan trọng nhất trong y tế. Accuracy thấp hơn (74.42%) do không thiên về lớp đa số. Precision thấp (14.29%) nghĩa là có nhiều cảnh báo sai nhưng đó là trade-off chấp nhận được trong y tế. Cross-validation ổn định (0.7400). Mô hình đơn giản, dễ diễn giải và **ưu tiên phát hiện ca bệnh**.

6.3 So sánh tổng hợp các mô hình



Hình 7: So sánh các chỉ số Accuracy, Precision, Recall, F1-Score giữa 4 mô hình



Hình 8: So sánh ROC-AUC và Cross-Validation scores giữa 4 mô hình

Kết luận so sánh:

- **Mô hình tốt nhất cho ứng dụng y tế: Logistic Regression** là lựa chọn tốt nhất với Recall cao nhất (82%) và ROC-AUC cao nhất (0.8438). Mô hình này phát hiện được phần lớn các ca đột quỵ, ưu tiên giảm false negatives - điều quan trọng nhất trong y tế.

- **Mô hình cân bằng:** SVM đứng thứ hai với Recall 66% và cân bằng tốt giữa các chỉ số. Phù hợp khi cần giảm cảnh báo sai (false positives).
- **Mô hình nhanh:** Naive Bayes phù hợp khi cần tốc độ phản hồi nhanh, mặc dù Recall chỉ đạt 34%.
- **Không khuyến nghị:** Random Forest không phù hợp trong cấu hình hiện tại do Recall = 0%, cần điều chỉnh ngưỡng quyết định hoặc áp dụng kỹ thuật resampling.

Khuyến nghị triển khai:

- **Ưu tiên 1:** Triển khai **Logistic Regression** cho hệ thống chính do Recall cao nhất.
- **Ưu tiên 2:** Sử dụng **SVM** như mô hình dự phòng hoặc khi cần cân bằng giữa Precision và Recall.
- **Cải thiện Random Forest:** Áp dụng SMOTE hoặc điều chỉnh ngưỡng quyết định để tận dụng ROC-AUC tốt (0.8159).
- **Đánh giá liên tục:** Theo dõi hiệu suất trên dữ liệu thực tế và điều chỉnh mô hình theo feedback từ bác sĩ.